Can a CNN Know When an Input Is Foreign? Benchmarking Four OOD Alarms on a Real Geographic Shift

Abdulrhman Bakr

October 2026

Abstract

Everybody is familiar with convolutional neural networks, but almost nobody asks them the one question they cannot answer: "is this input even from your world?" A CNN forced to classify a math equation will still answer "10% cat." In my previous research, a model that scored 93% on its European benchmark detected only 19% of real Amazon deforestation, because the Amazon was an out-of-distribution (OOD) world for it — and it never warned anyone. In this research I attach four known OOD warning functions to that same model — confidence, mumble (energy), center distance, and nearest-neighbor distance — and give them a real exam: separate 4,050 home images (the EuroSAT test split) from 289 foreign images (real Amazon patches), with every grading rule set before the results were seen. The result inverted my written predictions almost perfectly: the simplest baseline, the confidence function, won with a separation of 0.911, while the most reputable method, knn, came last at 0.775. Even the winner, at a threshold that leaves 95% of home images alone, caught only 54% of the foreign pile. The functions that read the model's internal map positions lost to the functions that read its verdicts — a result consistent with the idea that the network's own measurements hide the foreignness of foreign inputs. Not always do complex structures and algorithms win: this is now the second time in my research journey that the simplest method has beaten the sophisticated ones.

1 Introduction

Everybody is familiar with convolutional neural networks: they consist of neurons, weights, and layers. If I trained a CNN on dog and cat images and then, instead of giving it a picture of a dog or a cat to determine, I gave it a math equation, the model would still answer something like "10% cat." It has no way to say "this is not my field." The real question here: can the CNN really detect whether this input is a foreign input or not?

A foreign input is referred to as an OOD — out of distribution. The "distribution" is the model's training world, the whole spread of images it learned from; an OOD input is anything from outside that world. This is not a small academic detail. A self-driving model that meets a road situation it never trained on does not say "unsure" — it confidently labels it something and acts. A medical model trained on one hospital's scanner, deployed on another hospital's slightly different scanner, keeps producing confident, normal-looking answers while being systematically wrong. The failure is silent: a model that crashed would get checked, but a model that answers confidently gets believed.

I have personally produced one of these silent failures. In my last research, I trained a model to 93% accuracy on the EuroSAT benchmark (European satellite land patches) and deployed it on the Amazon to detect deforestation. It detected only 19% of the real forest loss, because the Amazon was an OOD world for it — it labeled

dense tropical jungle as "SeaLake" with confidence 0.83 — and at no point did anything warn that the inputs were foreign. That documented failure is what makes this research possible: I own a real, measured geographic shift, with the model, the imagery, and the ground truth all in hand.

The purpose here isn't to fully make the model detect OODs — that is an unsolved problem in the field; nobody has a reliable method, and the honest expert answer to "can a network know what it doesn't know?" is "we don't know how to do that." Instead, I aim in this research to do the two things that are actually possible with an unsolved problem: first, compare 4 known functions that detect OOD suspicion levels on my real shift and measure which of them actually works; and second, find the patches where the best-performing function failed and try to find why they failed, so that a targeted improvement can be aimed at a measured weakness rather than at a guess.

2 Background: how a CNN turns an image into a dot, and why it goes blind

To understand the four functions, we need to go back a little bit into the first steps of a CNN — because two of the functions read a structure most people never look at.

As we know, the inputted image is converted into pixels, and then, using matrices, the feature detectors (sometimes referred to as filters) are applied to this image. This extracts the information and features the model cares about: early filters catch edges and color changes, middle layers combine those into textures, late layers combine textures into concepts. After the filter layers, pooling summarizes everything into 2,048 measurements, where each measurement is a number describing one feature of the image — how leafy it is, how dark, how grid-like, and so on.

Here is the key jump: a list of numbers is a location. Two numbers make a point on paper; 2,048 numbers make a point in a 2,048-dimensional space — impossible to picture, trivial for math, same idea. So every image becomes a dot, and the space of all possible dots is the model's internal map. Similar images measure similarly, so they land near each other: all the forest training images pile up in one region, all the lakes in another.

The final layer of the network holds 10 scorer neurons, one per category. Each scorer owns 2,048 weights plus a bias; it multiplies the image's measurements by its own weights, adds its bias, and outputs one raw score. The highest score claims the dot. Borders between categories are never drawn anywhere — a border is simply the set of places where two scorers' totals tie, so the map is silently carved into 10 "countries." Softmax then converts the 10 raw scores into percentages that sum to 100%; the top percentage is what we call confidence, and it measures one thing only: how deep inside the winning country the dot landed — its margin from the border.

2.1 A worked miniature example

To make the scorers concrete, here is a baby version with only 2 measurements and 2 categories. Suppose the Forest scorer's weights are: leafy $+3$, dark -2 (it rewards leafiness and punishes darkness), and the SeaLake scorer's weights are the reverse: leafy -2, dark $+3$. Each scorer's weight list is like a wanted poster — the profile of measurements its category should have — and the bias is a head start added regardless of the input, which slides that category's border without tilting it.

A European forest arrives and measures leafy $= 0.9$, dark $= 0.1$. Forest score $= 3(0.9) - 2(0.1) = 2.5$; SeaLake score $= -2(0.9) + 3(0.1) = -1.5$. Forest wins, correctly. Now my Amazon jungle arrives, and the Europe-tuned instruments misread it: leafy $= 0.2$ (the tropical texture is unreadable to them), dark $= 0.9$. Forest score $= 0.6 - 1.8 = -1.2$; SeaLake score

$= -0.4 + 2.7 = +2.3$. SeaLake wins — my exact real error, reproduced in four multiplications.

Where is the border? It was never drawn: it emerges from the tie. Setting the two scores equal gives $3L - 2D = -2L + 3D$, so $L = D$ — the border is the line "leafy equals dark," and any dot with more leafy than dark belongs to Forest. Finally, softmax prices the winning margins: the forest photo's margin (2.5 vs -1.5) becomes about 0.98 confidence, and the jungle's margin (2.3 vs -1.2) becomes about 0.97 — confidently wrong, because the misread measurements placed the dot far from the border on the wrong side. This miniature is the entire anatomy of "SeaLake, 0.83."

Now the blind spot. The 10 countries tile the entire space; there is no "unknown" land anywhere on the map. So an input from outside the training world still gets measured, still lands somewhere, and still earns a verdict — and if the measuring instruments misread it, it can land deep inside the wrong country and earn high confidence there. That is exactly what happened to my jungle patch: the Europe-tuned filters read dense tropical canopy as "dark and flat," which is water's profile, so its dot landed deep inside SeaLake country, far from any border, and the model reported 0.83. Confident and alien at the same time, because confidence measures position relative to borders, never membership in the mapped world. Nothing in the machine ever asks whether the new dot is anywhere near the dots it trained on.

One more distinction matters for judging this research. A skin-cancer model shown a math worksheet is the easy case, called far-OOD: the worksheet lands nowhere near any skin photo, and crude checks catch it. My case is near-OOD: satellite imagery just like the training data, only from a different world, landing close to the home piles. Near-OOD is where every known method is weakest, and it is the case real deployments actually face — the different hospital scanner, the different continent. An exam on a real near-OOD shift is therefore the hard and honest version of the test.

3 Method

3.1 The four functions

Each function attaches to the existing trained model without retraining anything, and gives every image one danger score — higher means more suspicious. For all four functions the judging rule is the same: each function gives every image its own danger score; then a threshold line is set from the home pile so that 95% of home images pass as safe, and any image above the line is flagged. The threshold is set from the home pile only — we are allowed to know what home looks like, but we are never allowed to tune anything using the foreign pile, because in real life you do not have samples of the surprise before it arrives.

Function 1 — confidence. When our model outputs the 10 percentages (AnnualCrop, Forest, etc.), this function takes the highest percentage and flips it into a negative number, so an unsure image receives a higher danger score. A sure image (top percentage 0.95) gets danger -0.95; a torn image (top percentage 0.40) gets danger -0.40, which is higher. This method is the field's usual baseline — the thing every other method is supposed to beat. Its known weakness is the one I measured myself: foreignness does not guarantee unsureness, since an alien dot can land deep inside a country and score high confidence.

Function 2 — mumble. It takes the raw scores before the softmax function converts them into percentages, and then uses a torch function called logsumexp, which gives a "loudness" for each image. The reason to look before softmax: softmax only keeps the ratios between scores, so ten strong scores and ten weak scores can give nearly the same percentages — the overall loudness is erased. On a familiar image, one scorer shouts a big raw score; on a foreign image, often all ten scorers mumble. The lower the loudness, the more suspicious the image.

Function 3 — center. This function ignores the verdicts entirely and reads the map position.

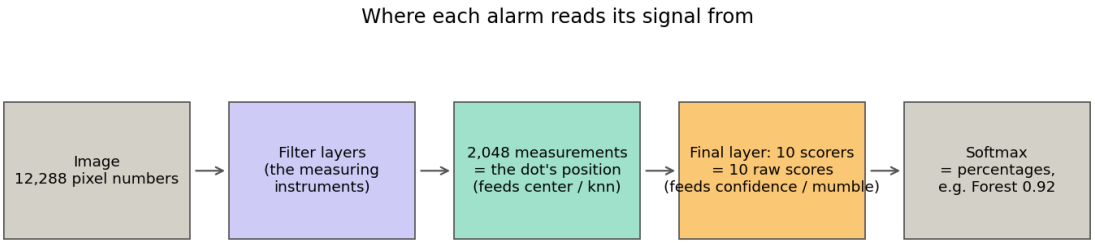


Figure 1: Where each alarm reads its signal from — the pipeline from pixels to percentages.

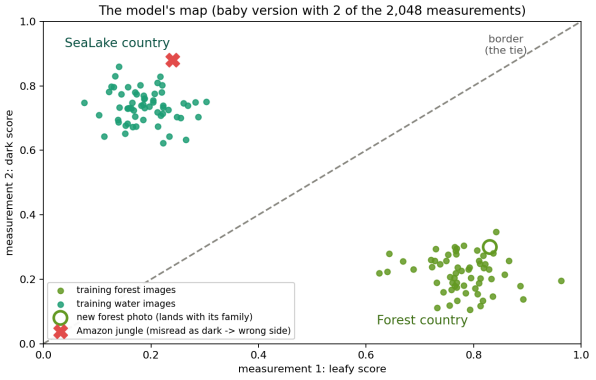


Figure 2: The map, baby version — two measurement axes, two training piles, the border, a new forest photo landing correctly with its family, and the misread jungle patch landing on the wrong side.

Each category's center is found as follows: for every training image of that category we take its 2,048 measurements and plot a dot, and then we average all of those dots, giving a center that sits in the middle of all that category's training dots. For a new point, we measure the distance from it to the center of every category — a smaller distance means it is more likely to be associated with that category. The center function outputs all the distances, then looks at the shortest one and asks: is even the shortest distance too far? If the answer is yes — even your best match is a bad match — then this is most likely an alarm-triggering dot.

Function 4 — knn. It has the same idea, the same map, and the same everything as the center function, but it fixes the center function's main weakness: a center is an average, and an average is a statistical representation made to satisfy all the training dots at once — which can place the center somewhere no real training image ever landed, and which wrongly flags legitimate images that sit at the far edge of a spread-out category. Knn does not depend on centers. It doesn't ask which center is at the shortest distance; it asks where the 5th-nearest real training neighbor is, and by how much — like asking "have I ever actually seen anything right here?" A big distance triggers the alarm and identifies the input as an OOD. Many would inquire: why the 5th neighbor and not the first or second? Because if 2 stray dots lie near each other but far from everything else, they could make the function wrongly stay calm; demanding five neighbors protects against such flukes, so 5 is a safer number.

3.2 The exam

We give each function two input piles. The home pile is the EuroSAT test split — 4,050 images from the model's own world that it was never trained on (using the same split seed as the original research, so no training image leaks into the exam); the functions must mark these as low danger. The foreign pile is the 289 Amazon patches cut from the real 2018 Sentinel-2 image of Rondônia, Brazil, used in my previous research — every one of these is foreign by construction, since the model trained only on European land, which means the OOD answer key costs nothing and involves no human labeling. We also save a bank of 2,000 training images; their plotted dots are what the center and knn

Table 1: The exam results for the four alarms.

alarm	separation	caught%
confidence	**0.911**	**54.0%**
mumble	0.854	49.5%
center	0.864	31.8%
knn	0.775	19.4%

functions measure their distances against.

To collect the signals, we cut the network before its final layer so it outputs the 2,048 measurements (the dot's position), then apply the final layer to those measurements separately to also record the 10 raw scores before softmax. Both are saved for every image — positions feed the center/knn alarms, raw scores feed the confidence/mumble alarms.

Each function is graded with two numbers. **Separation** is the probability that a randomly chosen foreign image receives a higher danger score than a randomly chosen home image: 0.5 means the alarm is a useless coin flip, 1.0 means perfect separation, and below 0.5 means the alarm points backwards. **Caught%** answers the practical question: with the threshold set so that 95% of home images pass as safe, what share of the foreign pile gets flagged?

3.3 Predictions, committed before testing

To keep the experiment honest, I wrote my predicted separation score for each function before any grading ran: confidence 0.55, mumble 0.62, center 0.75, knn 0.89. My reasoning followed the methods' reputations: confidence is "just the baseline," and knn is the refined memory-based method, so I expected the ranking to run from knn at the top down to confidence at the bottom.

4 Results

I tested all 4 functions, each using the input type it takes, and graded them. I was astonished. Table 1 shows the results.

The table clearly shows that the best-performing function was the confidence function — the simplest and easiest one, the one that is officially the field's baseline. Not always do complex structures and algorithms win.

Finding 1 — the ranking inverted the predictions. My written predictions were almost perfectly upside-down: I expected knn to be the strongest (0.89) and confidence to be among the weakest (0.55), while reality placed confidence first (0.911) and knn last (0.775). The committed predictions turn this reversal into a measured result rather than a hindsight story: the field's reputational ranking, and mine, ran backwards on this real geographic shift.

Finding 2 — the verdict readers beat the map readers. The two winning functions (confidence, mumble) read the model's verdict machinery — its percentages and raw scores. The two losing functions (center, knn) read the model's map positions — the 2,048 measurements. If the map-based alarms lose, the likeliest reading is that the network's own measuring instruments placed many Amazon dots close to the home piles: the map itself hid the foreignness, leaving the distance rulers little to measure, while the collapse in confidence survived as the louder signal. This is consistent with (though it does not prove) a known deep concern in the field: training rewards the filters for keeping only what separates the 10 categories and discarding everything else, so the evidence of foreignness may be partially erased from the measurements before any distance check gets to read them.

Finding 3 — the crude ruler beat the refined one. Inside the map-reading family, the primitive center function (0.864) outperformed its own supposedly superior refinement, knn (0.775) — a second reputation inversion nested inside the first. I record this as an open question for the failure investigation rather than explain it prematurely.

The honest asterisk. A separation of 0.911 sounds heroic, but look at caught%: at a threshold that leaves 95% of home images alone, even the winner flags only 54% of the foreign pile.

Nearly half the Amazon walks past the best alarm. So the headline is double-edged: the baseline wins, and even the winner misses almost half. No alarm in this table is ready for real deployment; the exam measured exactly how far from ready they are. The gap between the two grades is natural: separation grades the overall ordering of the two piles, while caught% asks about the strict-threshold regime where the piles' overlap bites.

4.1 The histogram figure, explained part by part

Figure 3 contains four plots, one per function. In each plot, the x-axis is the danger score that function assigns, and the y-axis is how many images received that score. The blue shape is the home pile (the 4,050 EuroSAT test images) and the orange shape is the foreign pile (the 289 Amazon patches). The one rule for reading them: what matters is the horizontal overlap between the two shapes, never their height or their beauty. Height only shows how concentrated the scores are; horizontal position is the signal. A perfect alarm would show blue and orange standing completely apart; a useless alarm shows them sitting on top of each other.

Confidence (separation 0.911). The blue forms a tall needle at -1.0, because nearly every home image receives close to maximum confidence — the model is extremely sure on its own world, so home occupies one narrow spot. The orange is a low, wide spread from about -0.9 to -0.2, because foreign confidences collapse to many different lower values. Almost all of the orange area lies to the right of the blue needle, so the horizontal overlap is tiny — which is exactly why this plot carries the best separation despite the orange hump looking small and flat. The thin orange tail that hides under the needle, around -1.0 to -0.9, is the most important detail in the whole figure: those are foreign patches that scored near-maximum confidence — the confidently-wrong population, the same species as the "SeaLake 0.83" jungle patch from my previous research — and they are precisely the sleeper patches the next phase investigates.

Mumble (0.854). Both humps are clearly visible, and the orange peak sits to the right of the blue one, but the two share a wide middle zone where foreign and home images receive the same loudness values; that shared zone is what costs it against confidence.

Center (0.864). A similar picture — the orange hump is shifted right of the blue, but a substantial overlap region remains where foreign patches sit at home-like distances from the class centers.

Knn (0.775). The blue and orange sit on top of each other through most of the range — the heaviest horizontal overlap of the four, which is why it scored worst. Foreign dots are landing at distances from their nearest training neighbors that are barely different from the distances at which home images land: on the model's internal map, much of the Amazon does not look far from home.

The caught% is also visible in each plot: imagine a vertical line at the right edge of the blue shape, placed so 95% of the blue area is to its left — whatever portion of the orange area sticks out to the right of that line is the caught share, and the orange mass hiding to the left of it, inside blue territory, is the foreign patches that walk past the alarm. A note on the visual trap this figure sets: the mumble, center, and knn plots look "nicer" because both humps are tall and visible, but tall-and-visible-while-overlapping is exactly what a weak alarm looks like; the eye grades aesthetics, the separation number grades overlap, and that is why we compute instead of squint.

5 Discussion

Would an attached alarm have prevented my previous disaster? Partially — and the nuance matters. The confidence alarm catches 54% of foreign patches individually, which sounds mediocre; but a deployment is not one patch. If

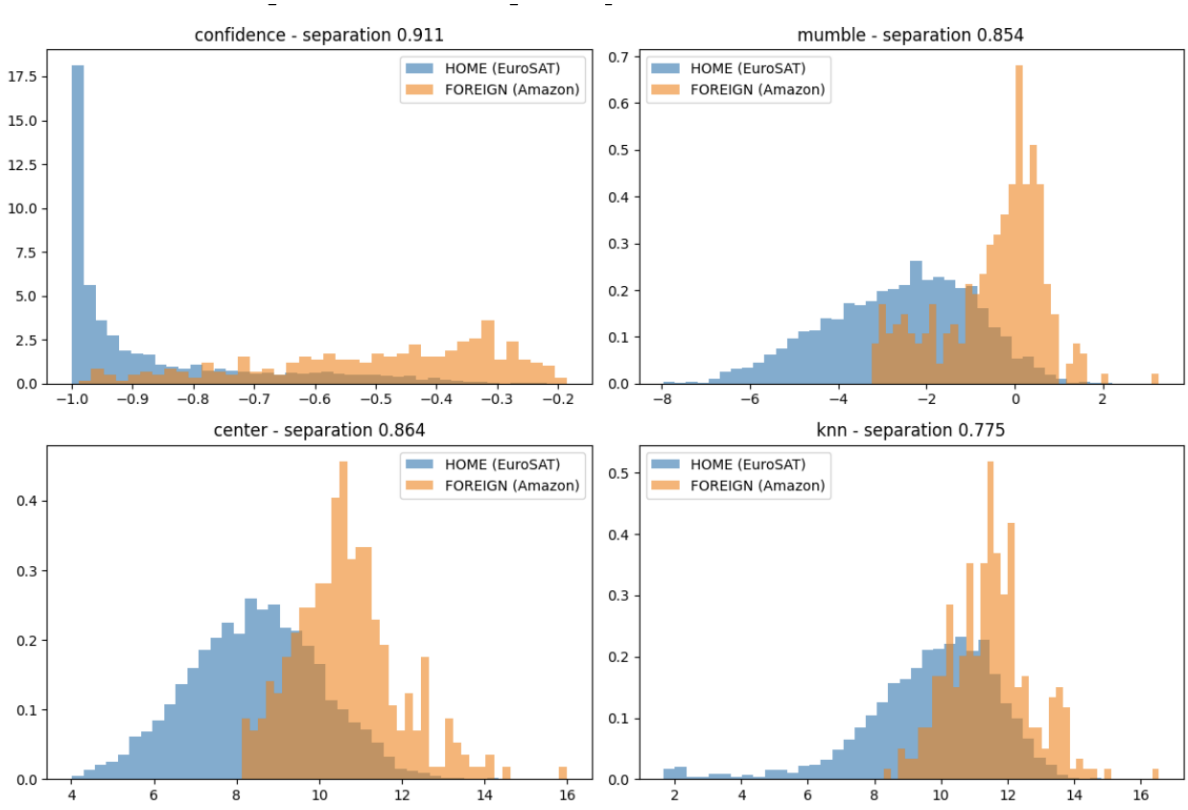


Figure 3: The four alarms' danger scores — home pile (blue) vs foreign pile (orange), one plot per function, with each plot's separation in its title.

more than half of an incoming region's patches ring the alarm, the region as a whole screams — any operator watching an alarm rate that jumps from the expected 5% to over 50% knows immediately that the model has left its world. So at the region level, even this imperfect alarm would likely have flagged the Amazon deployment before the 19%-recall failure was trusted; at the individual-patch level, nearly half the foreign inputs would still have been silently accepted. Per-input OOD detection and per-deployment OOD detection are different strengths of the same claim, and this table only directly certifies a useful version of the weaker one.

Why might the map readers have lost? The measurements that the center and knn functions read are produced by instruments trained for one purpose: separating the 10 training categories. Everything that does not help that separation tends to be compressed away — and "this texture is unlike anything from training" is exactly such information. If the instruments partially erase the evidence of foreignness, then the dots of foreign images land close to home piles, distances stay small, and distance-based alarms starve — while the scorers' hesitation (lower confidence, quieter raw scores) remains detectable downstream. My table cannot prove this mechanism, but it is the simplest story consistent with the verdict readers beating the map readers, and it is checkable: measuring the distances on earlier layers, where the instruments are less specialized, is a concrete follow-up experiment.

The recurring pattern. This is the second time my own experiments crowned the simplest method: in my first research, plain multiple linear regression beat support vector regression and random forests on messy tabular data; here, the confidence baseline beat three more sophisticated detectors on a real shift. Reputations are built on benchmark conditions; real data keeps electing the boring candidate. I now treat "test the simplest thing first, and in ink" as a standing rule rather than a surprise.

6 Limitations

This study tests one model, one training world, and one foreign region — a single Europe-to-Amazon shift; the ranking could differ on other kinds of shift, and no claim is made beyond this setting. The foreign pile is small (289 patches from one 2018 composite of one 11×11 km rectangle), while the home pile is large (4,050 images), so the foreign side of the exam carries more statistical noise. The distance alarms measured against a bank of 2,000 training images rather than all 18,900, which makes knn's distances slightly larger and noisier than they could be. The center function used plain straight-line distance, not the shape-aware version used in the strongest published work, so this table compares simple implementations, not the methods' best possible forms — a deliberate choice, since the goal was to test what a practitioner can cheaply attach to an existing model. The knn neighbor count ($k = 5$) was fixed in advance and not tuned. The home and foreign piles also differ in processing history (EuroSAT's prepared tiles versus my own Earth Engine composites), so a small part of the separation could come from processing differences rather than geography alone. Finally, the two grades depend on the chosen operating point: the 95% home-pass threshold is a convention, and a different threshold would change caught% (though not separation).

7 Conclusion

Four known OOD warning functions took a real exam on a real geographic shift with the answer key fixed in advance. The simplest baseline — reading the model's own confidence — won with a separation of 0.911, inverting both the field's reputational ranking and my own written predictions, while the most reputable method, nearest-neighbor distance on the model's internal measurements, came last at 0.775. The functions that read the model's verdicts beat the functions that read its internal map, consistent with the measurements themselves hiding the foreignness of foreign inputs. And even the winning alarm, constrained to leave 95% of home images alone, caught only 54% of the foreign pile — so the honest summary is that on a real near-OOD shift, the best cheap warning light works at half strength and the sophisticated ones work less. A model that scores 93% at home can go abroad, fail, and the standard alarms will sleep through much of it. Not always do complex structures and algorithms win; measuring which simple thing actually works, before trusting any reputation, is the entire lesson of this exam.

8 Future Work

The next phase is already defined by this table. First, the sleeper investigation: pull the foreign patches that the winning confidence function was calmest about — the orange tail hiding under the blue needle — look at them by eye, and categorize what they share; that shared property is the measured jam. Second, one targeted change aimed at that exact jam — for example, reading the distance alarms from an earlier layer, where the measurements are less specialized and may erase less of the foreignness, or combining a verdict-reader with a map-reader where their failures do not overlap — re-run on the exact same exam, and report before versus after against the 0.911 baseline. Further ahead: repeat the exam on other shifts (other regions, other seasons, other sensors) to test whether confidence's win is general or a property of this shift, and sweep the k dial and bank size for knn to check how much of its loss is implementation rather than principle.

Acknowledgment

AI assistance was used for debugging code, verification of results, and language editing. Datasets and models: EuroSAT (Helber et al., 2019), Sentinel-2 imagery via Google Earth Engine (the Amazon patches reuse the composites from my previous deforestation study),

ResNet50 pretrained on ImageNet. All experiments, decisions, predictions, and findings are the author's own.

Table 2: Glossary of the terms used in this paper.

Term	Plain meaning
Distribution	The model's training world — the whole spread of data it learned from
OOD (out of distribution)	An input from outside that training world
Near-OOD / far-OOD	A subtly different world (jungle to a Europe model) / an obviously alien input (a math sheet to a skin model)
The 2,048 measurements	The numbers the filter layers produce for an image — its description sheet, and its coordinates on the internal map
The map	The 2,048-dimensional space where every image is a dot; similar images land near each other
Scorer / raw score (logit)	One output neuron per category; its weighted sum of the measurements, before softmax
Border	The set of points where two scorers tie — where the verdict flips; never stored, always implied
Softmax	Converts the 10 raw scores into percentages summing to 100%
Confidence	The top percentage — the winning margin; depth inside the winning country, not correctness
Danger score	The one number per image that each alarm outputs; higher = more suspicious
Home / foreign pile	EuroSAT test images (inside the world, never trained on) / the 289 Amazon patches (outside it by construction)
Bank	2,000 saved training images whose dots the distance alarms measure against
Separation	Probability a random foreign image out-scores a random home image in danger; 0.5 coin flip, 1.0 perfect
Caught%	Share of the foreign pile flagged when the threshold lets 95% of home pass as safe
Sleepers	Foreign patches the best alarm was calmest about — the next phase's targets